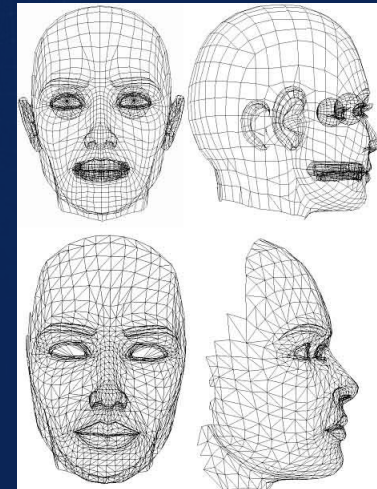
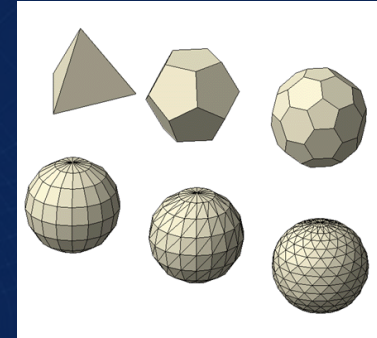


Vorlesung: Grundlagen der Computergrafik

Kapitel 3: 3D-Computergrafik

Kapitel 3.5: Polygonnetze



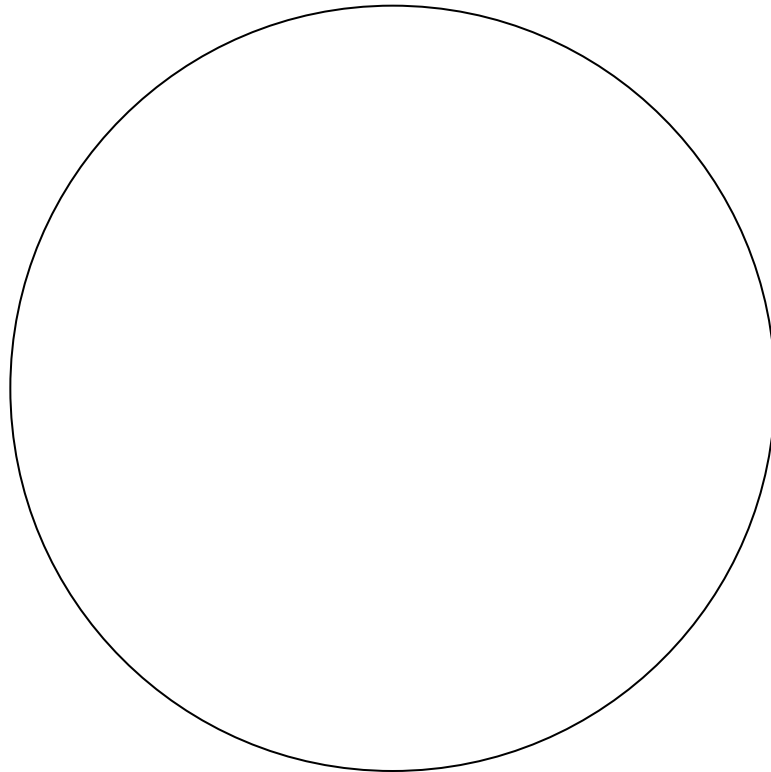
Vorlesung im Sommersemester 2026

Prof. Dr. Stephan Diehl, Informatik, Universität Trier



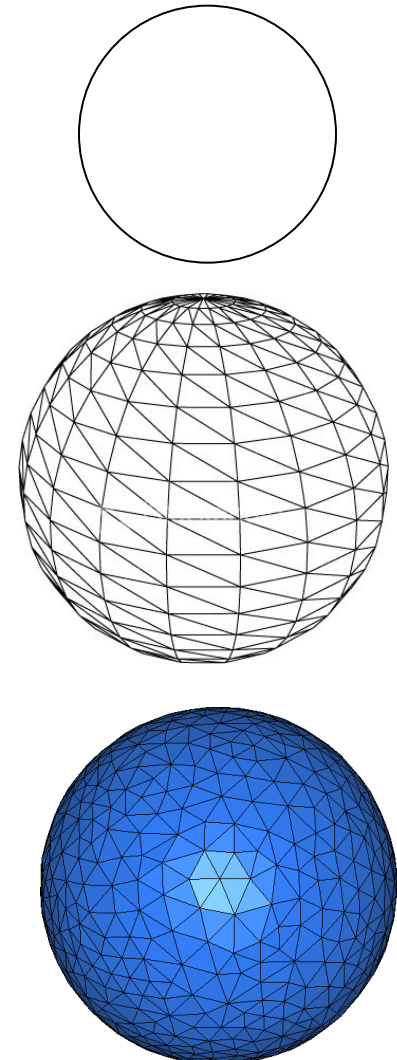
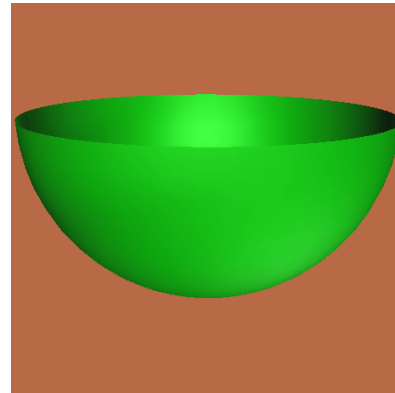
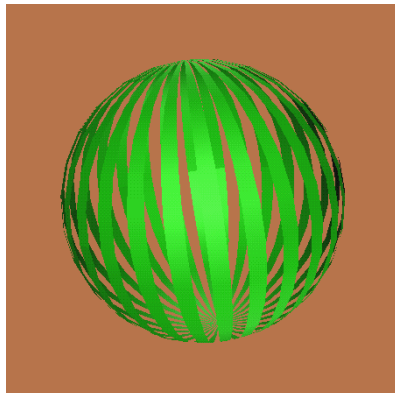
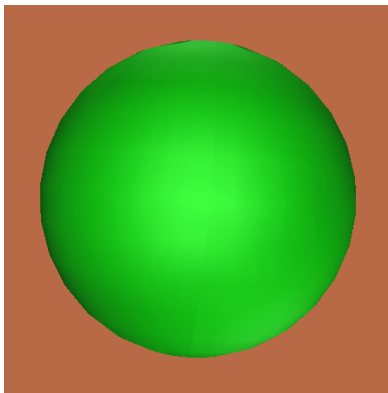
Randdarstellung (Boundary Representation)

- Zweidimensionale Objekte (Flächen) werden durch eindimensionale „Ränder“ beschrieben.



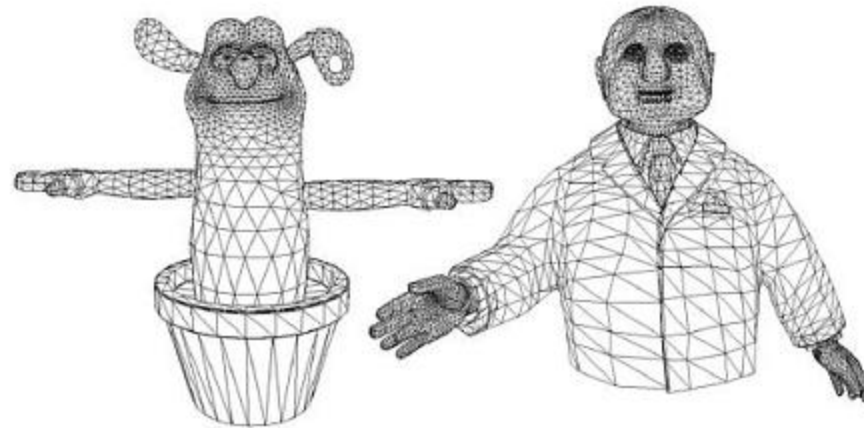
Randdarstellung (Boundary Representation)

- Zweidimensionale Objekte werden durch eindimensionale „Ränder“ beschrieben.
- Dreidimensionale Objekte durch zweidimensionale „Ränder“, also Flächen.
- Was passiert beim „Aufschneiden“?
- Sind Objekte wirklich hohl?



Polygonnetze

- einfachste Form der Randdarstellung für 3D
- Beschreibung eines Objektes durch eine Menge von planaren Polygonen (insbesondere Dreiecksnetze)



Polygonzug

Seien $V_0, \dots, V_{n-1}$ Punkte im zweidimensionalen oder dreidimensionalen Raum.

$Q = \{(V_0, V_1), (V_1, V_2), \dots, (V_{n-2}, V_{n-1})\}$ ist ein **Polygonzug**. Die Paare sind die **Kanten** des Polygonzugs. Die umrandete Fläche nennen wir **Polygon**.

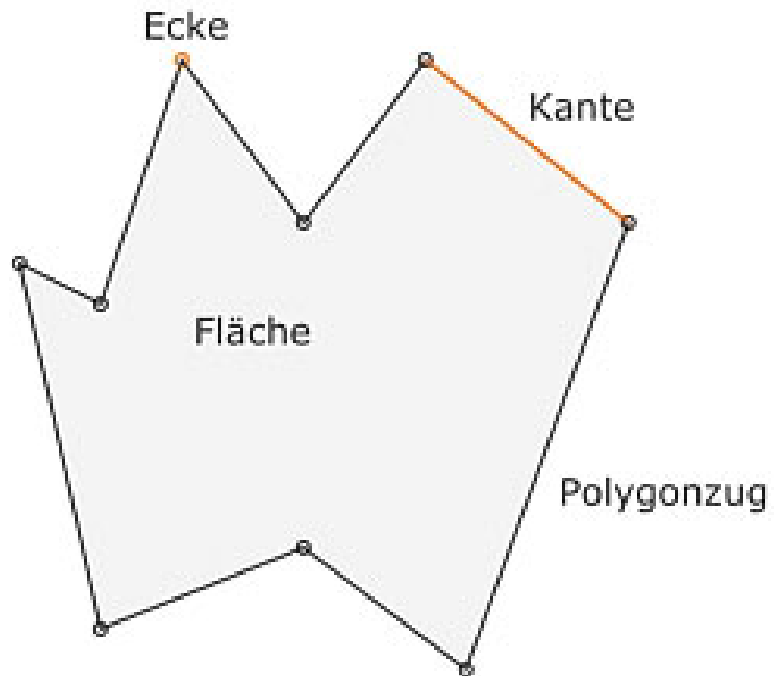
Ist $V_{n-1} = V_0$, dann ist der Polygonzug **geschlossen** und die Anzahl der Eckpunkte ist gleich Anzahl der Kanten.

Liegen alle Kanten in einer Ebene, dann ist das Polygon **planar**.

Ein Polygon ist **einfach**, wenn sich keine zwei Kanten schneiden (außer in einem Eckpunkt) und jeder Eckpunkt gehört höchstens zu zwei Kanten.

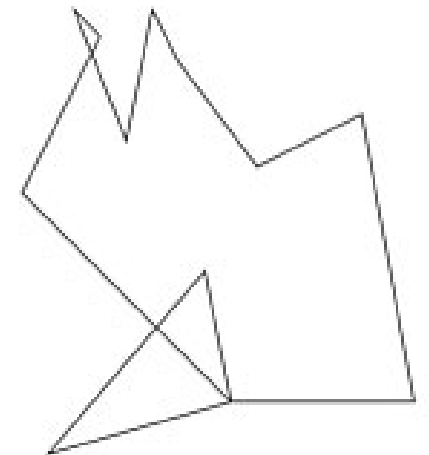
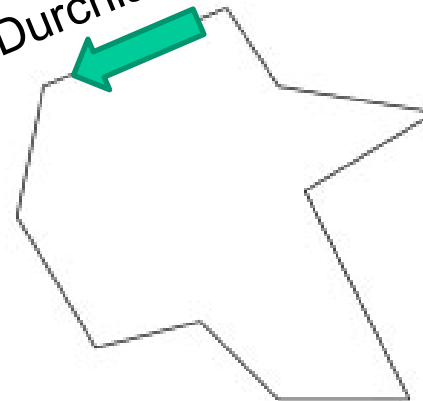
Positive Durchlaufrichtung (Gegenuhrzeigersinn):
das Innere des Polygons ist links der Kante.

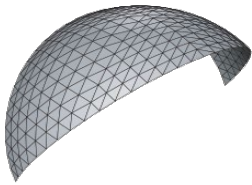
Polygon



einfaches Polygone nicht einfaches Polygone

Positive
Durchlaufrichtung

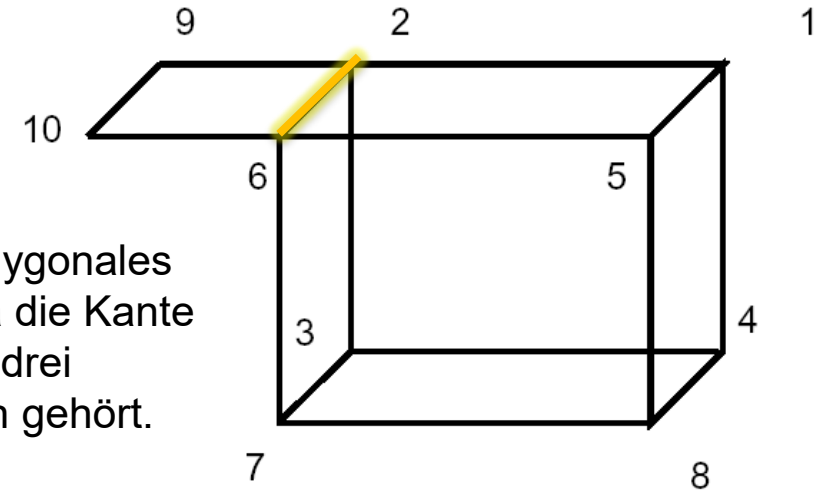




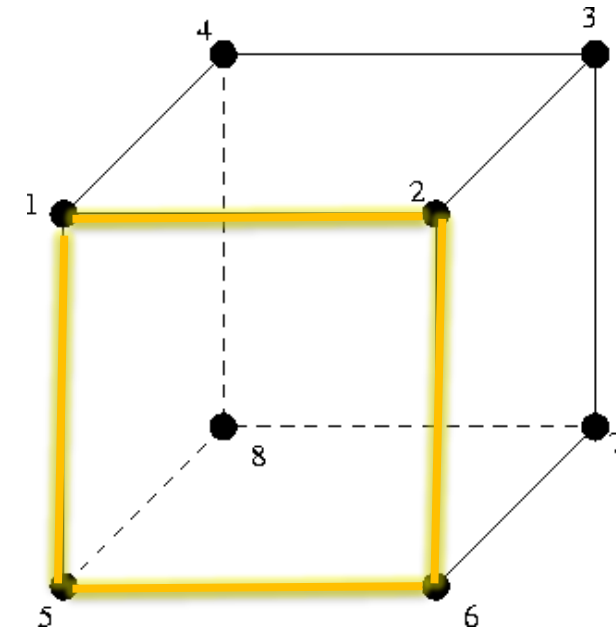
Polygonnetze

Polygonnetz = Menge von geschlossenen, planaren und einfachen Polygonen (Facetten) und:

- Der **Schnitt von zwei Facetten** ist entweder leer, ein Eckpunkt oder eine Kante ($\Rightarrow$ keine überlappende Fläche)
- **Jede Kante** einer Facette gehört zu höchstens zwei Facetten.
- Die Menge aller Kanten, die nur zu einer Facette gehören, ist leer oder lässt sich partitionieren, so dass jede Teilmenge einen geschlossenen, einfachen Polygonzug bildet (**Rand/Loch des polygonalen Netzes**). Ist die Menge leer, dann ist das Netz geschlossen.

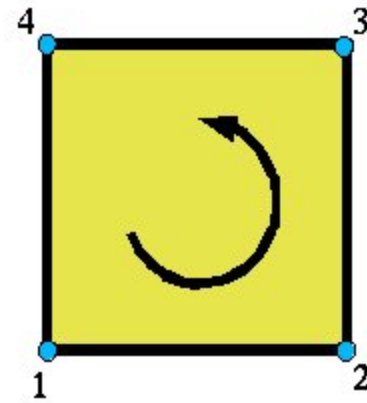


Kein Polygonales Netz, da die Kante (6,2) zu drei Facetten gehört.



Falls z.B. die Facette (1,5,6,2) nicht im polygonalen Netz enthalten ist, bilden die übrigen Facetten ein offenes polygonales Netz. Die roten Kanten bilden den Rand.

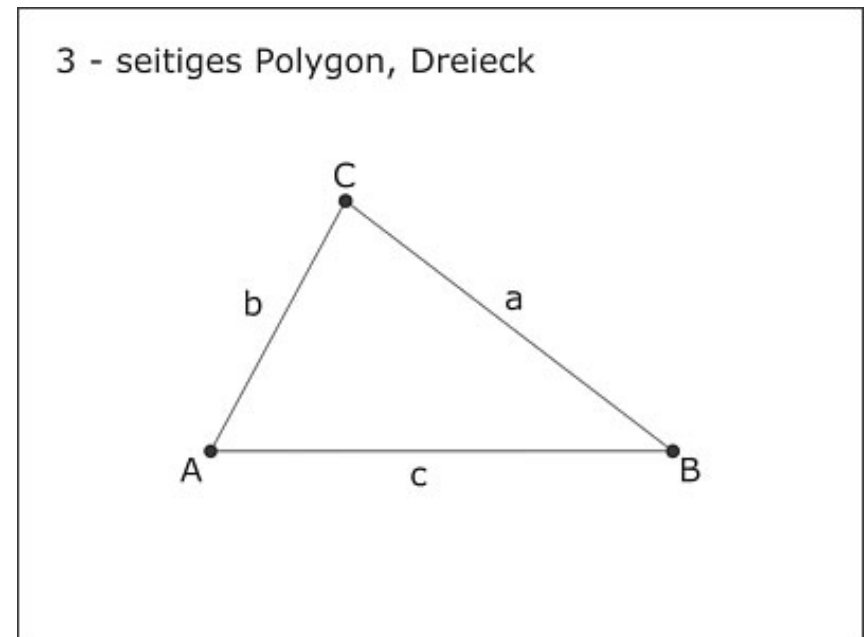
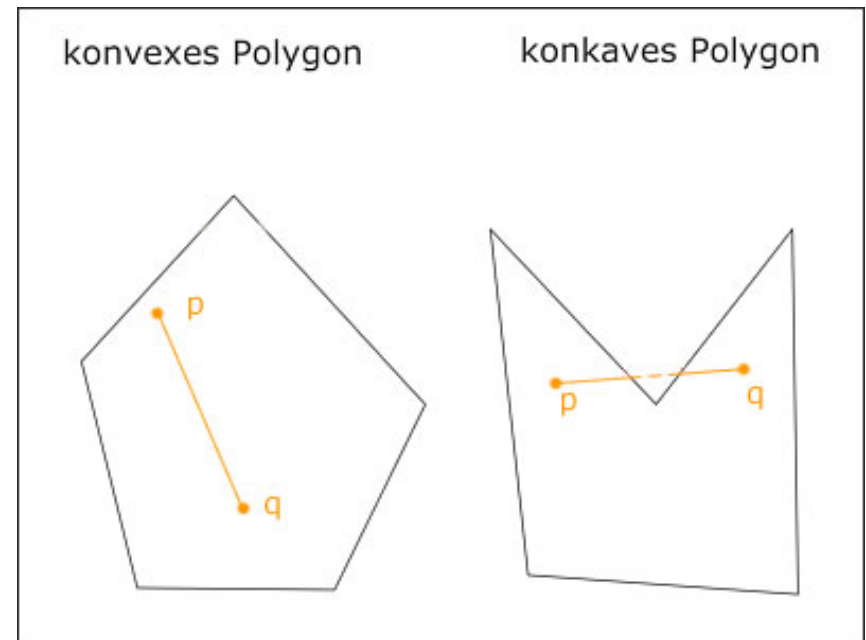
Durchlaufrichtung



- Eine Facette hat eine Vorder- und eine Rückseite.
- Konvention: Wir sehen die Vorderseite der Facette, falls der Rand der Facette **gegen den Uhrzeigersinn** durchlaufen wird!
- In vielen Software-Paketen wird so die Entscheidung getroffen, was ein **front face** und was ein **back face** ist!

Polygonnetze

- Häufig: Einschränkung auf Netze mit konvexen Polygonen
- Ein Polygon ist **konvex**, wenn für zwei beliebige Punkte V und W des Polygons alle Punkte $P = (1-t)*V + t*W$ mit $t \in [0,1]$ im Polygon liegen.
- Das Polygon mit der kleinsten Anzahl an Ecken und Kanten, das bereits eine Fläche umfasst, ist das dreiseitige Polygon oder **Dreieck**.

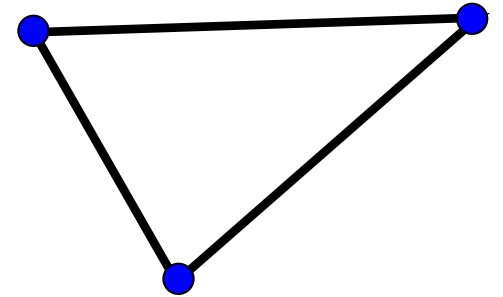


DATENSTRUKTUREN FÜR POLYGONNETZE

Topologie vs. Geometrie

- **Topologie:** die Menge der Eigenschaften des Objektes, die durch **stetige, bijektive** Abbildungen des Raumes (z.B. Strecken, Stauchen, Verdrehen), nicht verändert werden – die Struktur des Modells.

Im Beispiel: Das Polygon ist ein Dreieck.



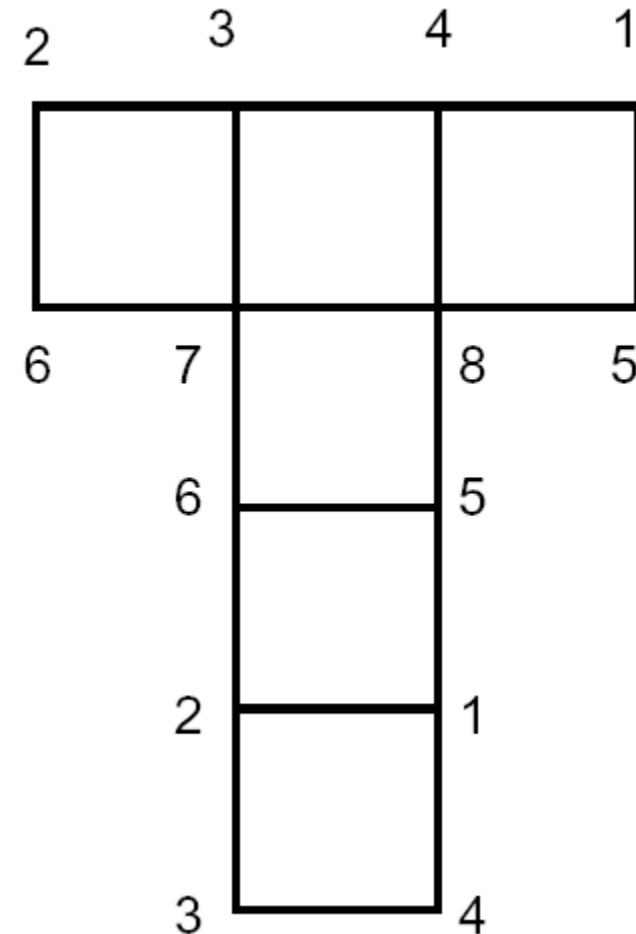
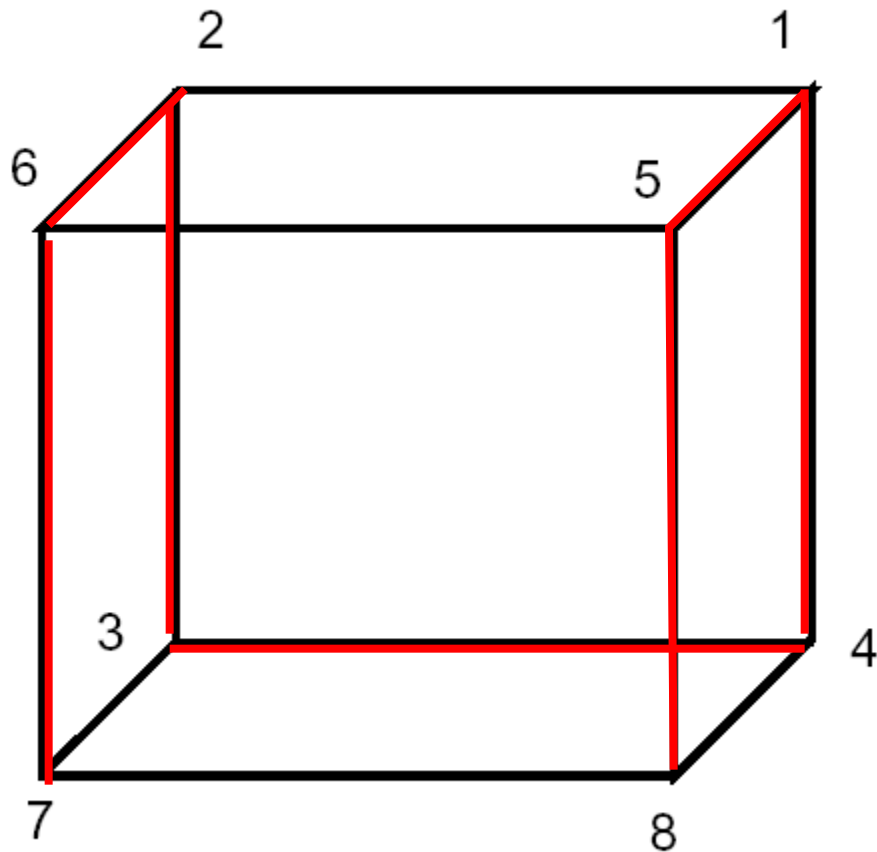
- **Geometrie:** die „Instanziierung“ der Topologie durch Spezifikation der räumlichen Lage - die Form des Modells.

Im Beispiel: Die Koordinaten des Dreiecks

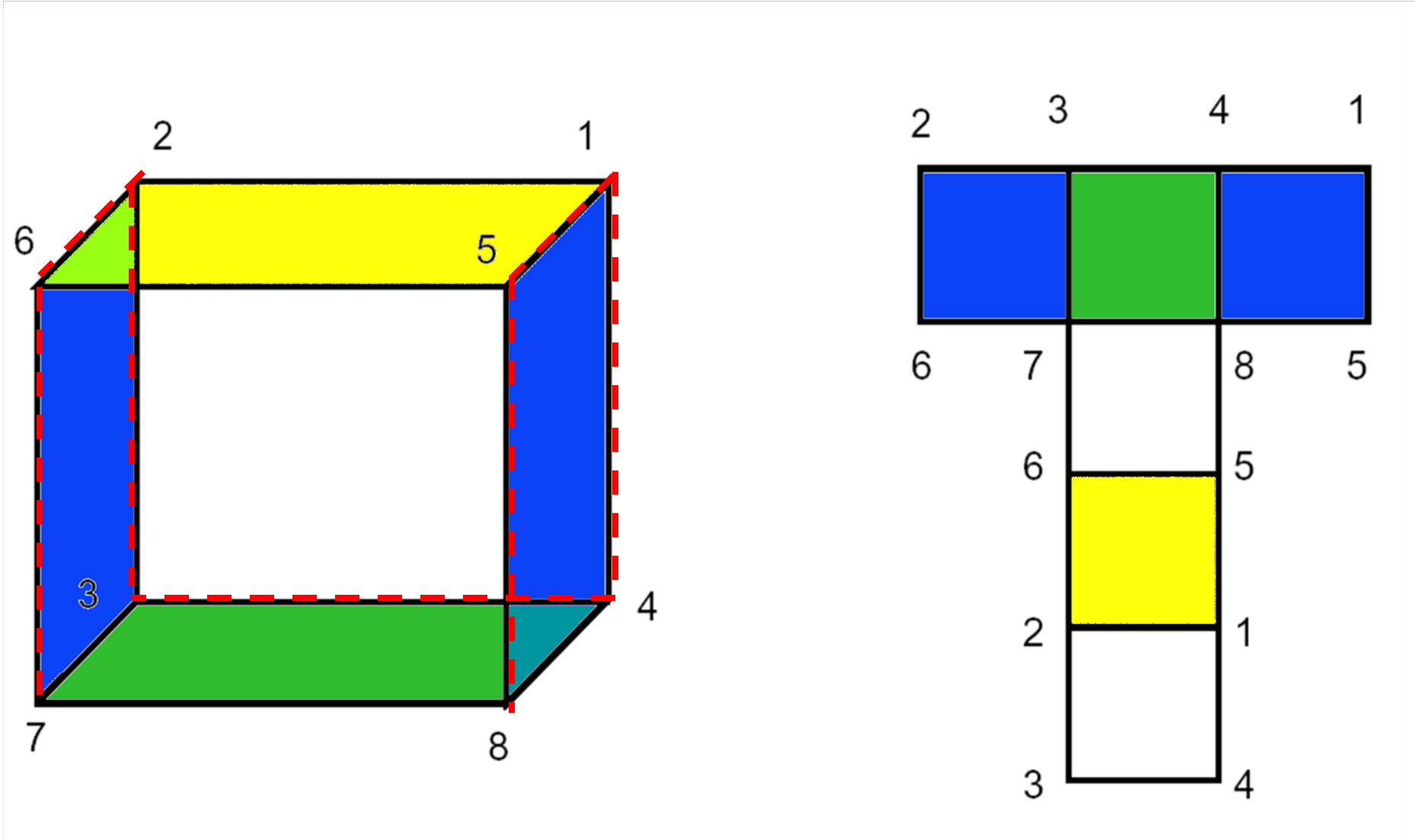
Baupläne

- = „Grundriss“ des Netzes.
- Aufschneiden des Netzes entlang von Kanten und Ausrollen in der Ebene.
- Dadurch entsteht ein **planarer Graph**, d.h. ein Graph, der ohne Kantenkreuzungen gezeichnet werden kann.

Bauplan eines Würfels

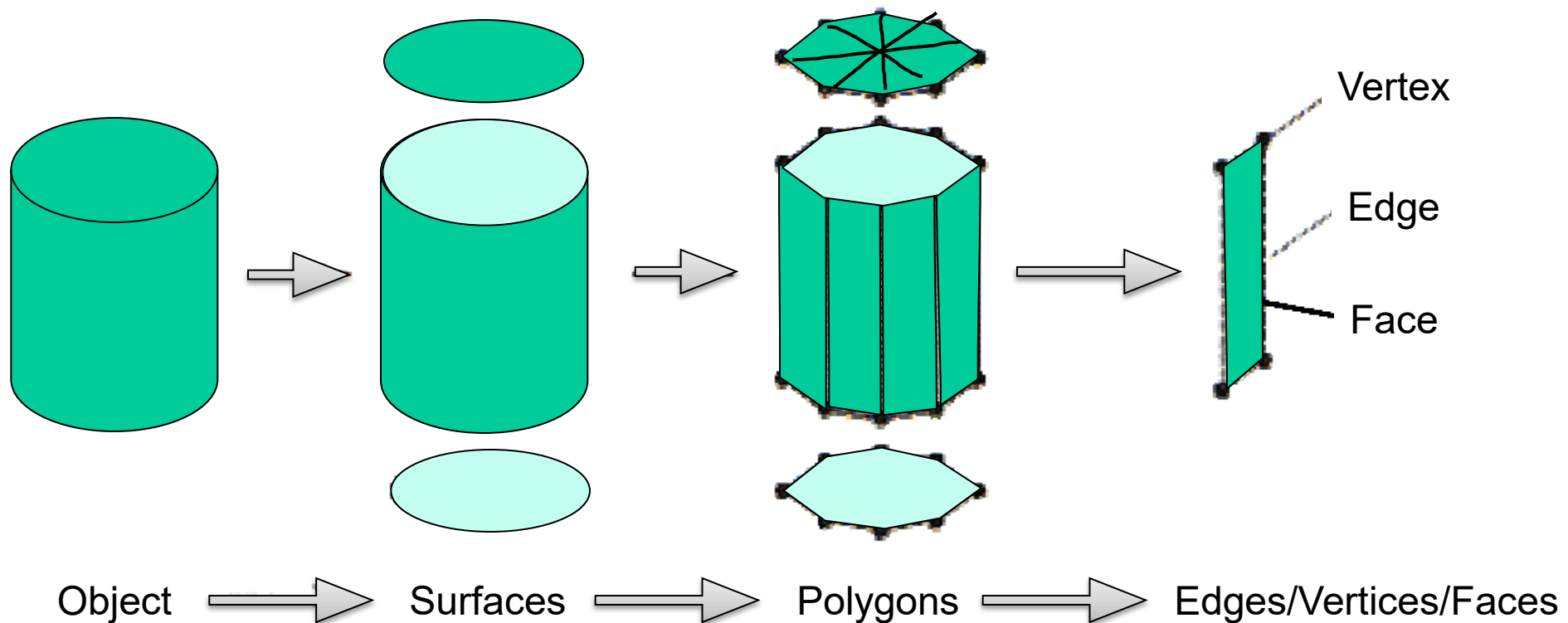


Bauplan eines Würfels



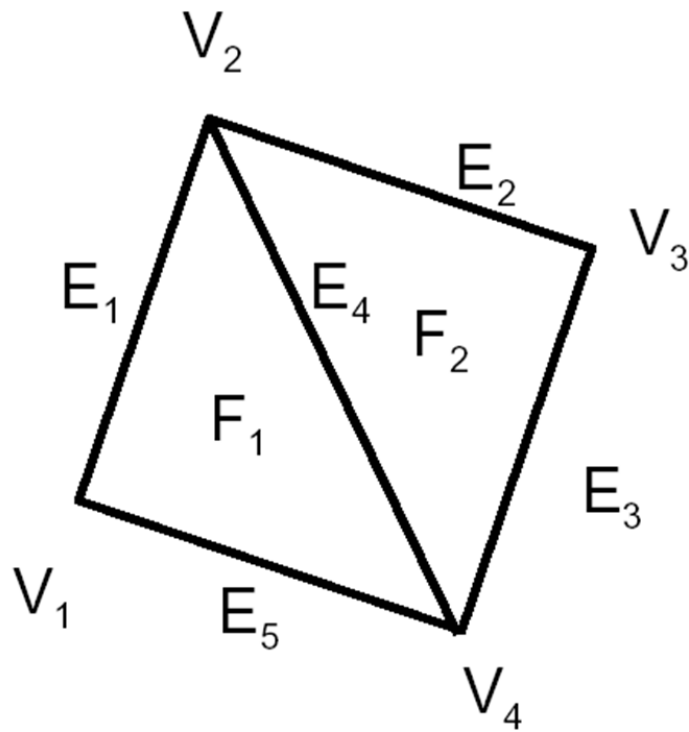
Datenstrukturen für polygonale Netze

- Datenstrukturen zur effizienten Speicherung und Bearbeitung großer polygonaler Netze.



Datenstrukturen für polygonale Netze

- Wir müssen sowohl die **Topologie** als auch die **Geometrie** explizit speichern.
- Das Problem ist die Topologie, die Geometrie besteht aus einer Liste von Punktkoordinaten!



Eckenliste

- $V = (V_1, V_2, V_3, V_4)$
- $F_1 = (1, 4, 2)$
 $F_2 = (4, 3, 2)$

Explizite Darstellung

- $F_1 = ((V_{1x}, V_{1y}, V_{1z}), (V_{4x}, V_{4y}, V_{4z}), (V_{2x}, V_{2y}, V_{2z}))$
- $F_2 = ((V_{4x}, V_{4y}, V_{4z}), (V_{3x}, V_{3y}, V_{3z}), (V_{2x}, V_{2y}, V_{2z}))$

Kantenliste

- $V = (V_1, V_2, V_3, V_4)$
- $E_1 = (V_2, V_1, F_1, N)$
 $E_2 = (V_3, V_2, F_2, N)$
 $E_3 = (V_4, V_3, F_2, N)$
 $E_4 = (V_4, V_2, F_1, F_2)$
 $E_5 = (V_1, V_4, F_1, N)$
- $F_1 = (E_1, E_5, E_4)$
 $F_2 = (E_3, E_2, E_4)$

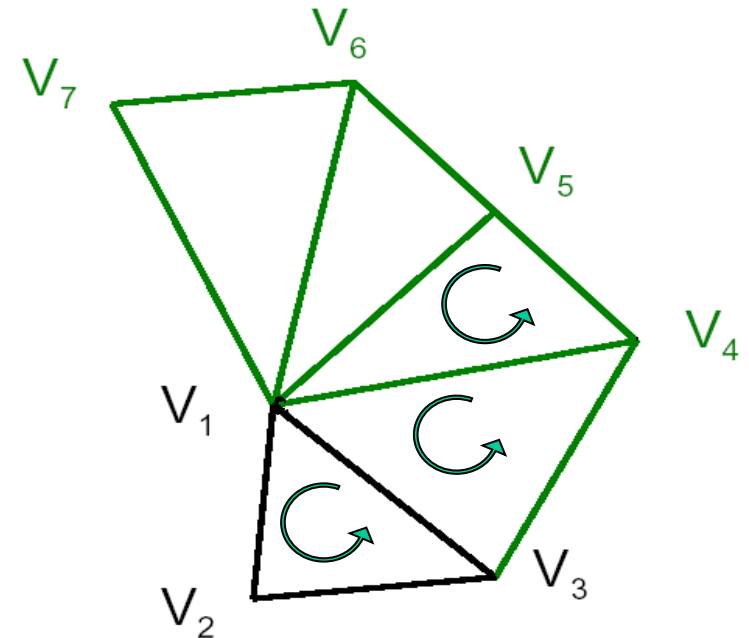
Dreiecksnetze

- Dreiecke sind immer planar.
- Ecken- oder Kantenliste als Datenstruktur erfordert für n Dreiecke **$3 \cdot n$** Eckpunkte (oder Zeiger auf Eckpunkte) beim Speichern oder Übertragen an die Rendering-Pipeline

Triangle Fan

- Ein Triangle Fan ist eine Folge von mindestens drei Eckpunkten ($V_1, V_2, V_3, \dots$).
- Insgesamt definieren **$n+2$** Eckpunkte n Dreiecke.
- Je zwei aufeinander folgende Eckpunkte definieren mit dem ersten Eckpunkt ein Dreieck:

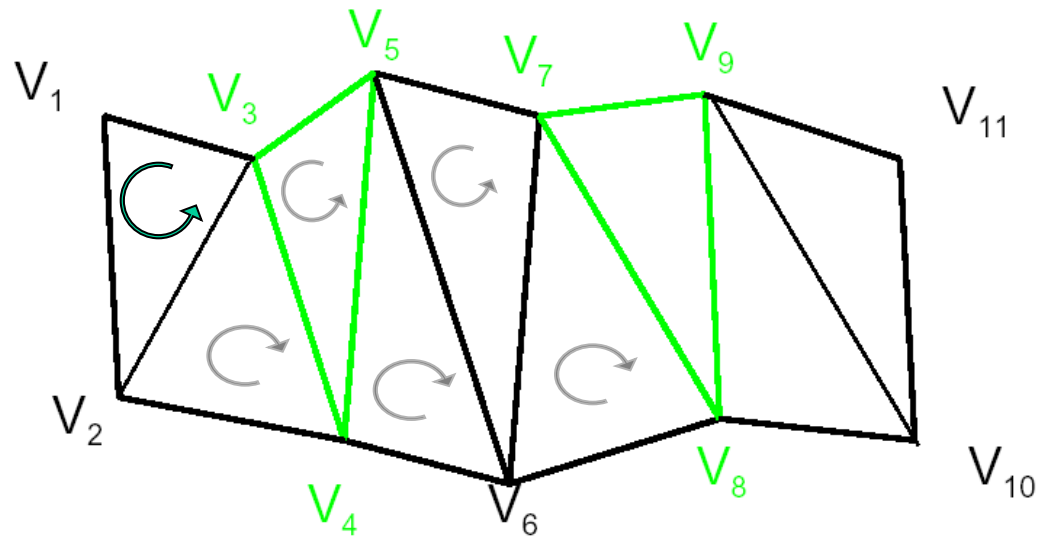
$(V_1, V_2, V_3, V_4, V_5, V_6, V_7)$



Triangle Strip

- Ein Triangle Strip ist eine Folge von mindestens drei Eckpunkten ($V_1, V_2, V_3, \dots$).
- Insgesamt definieren **$n+2$** Eckpunkte n Dreiecke.
- Je drei aufeinander folgende Eckpunkte definieren ein Dreieck:

($V_1, V_2, V_3, V_4, V_5, V_6, V_7, V_8, V_9, V_{10}, V_{11}$)



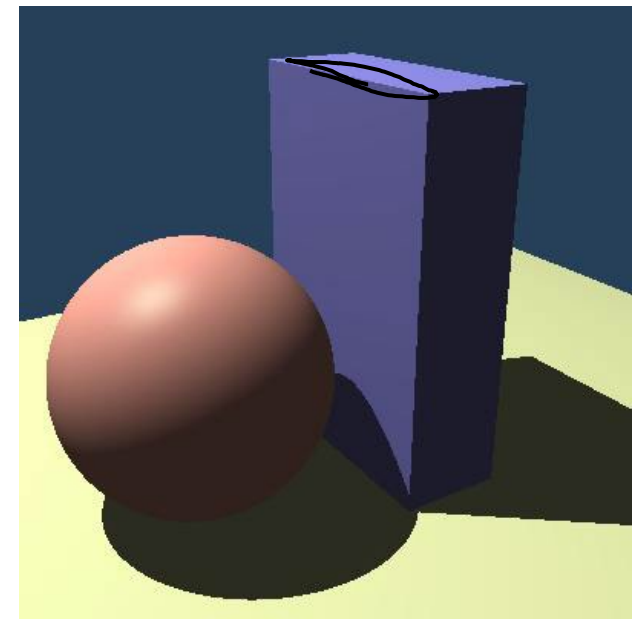
Triangle Strips & Triangle Fans

- Implizite Topologiespeicherung
- Geringer Speicheraufwand (ein Drittel)
- Bessere Performance auf der Graphik-Hardware
- Die Graphik-APIs OpenGL, Direct3D und Java3D unterstützen Strips und Fans.

Eckenliste		Strip oder Fan	Verlgeich
n	3*n	n+2	(n+2)/(3*n)
1	3	3	1
4	12	6	0,5
10	30	12	0,4
100	300	102	0,34
1000	3000	1002	0,334
10000	30000	10002	0,3334
100000	300000	100002	0,33334

Kodierung echter Kanten

- Offensichtlich existieren in der approximierenden polygonalen Darstellung zwei Arten von Kanten:
 - Kanten, die auch in Wirklichkeit existieren
→ diese sollen als Kanten sichtbar bleiben
 - Kanten, die in der Realität nicht existieren
→ diese sollte der Renderer „verschwinden“ lassen (Schattierungsalgorithmen)
- Die Art der Kanten wird durch die Datenstruktur bestimmt, z. B. durch die Mehrfachspeicherung von Ecken oder Kanten.



Smoothing

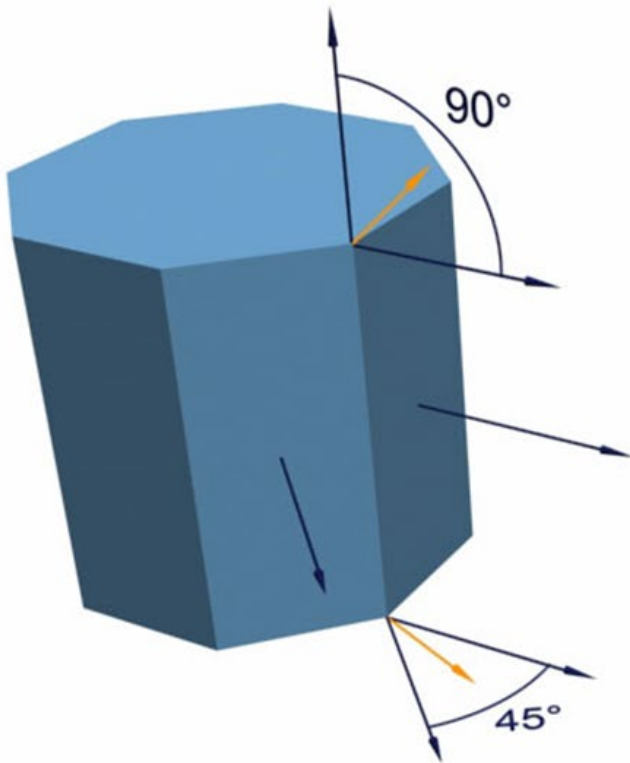
- Aus zwei benachbarten Flächennormalen wird ein Normalvektor errechnet, der an die Kante der beiden Faces verschoben wird.



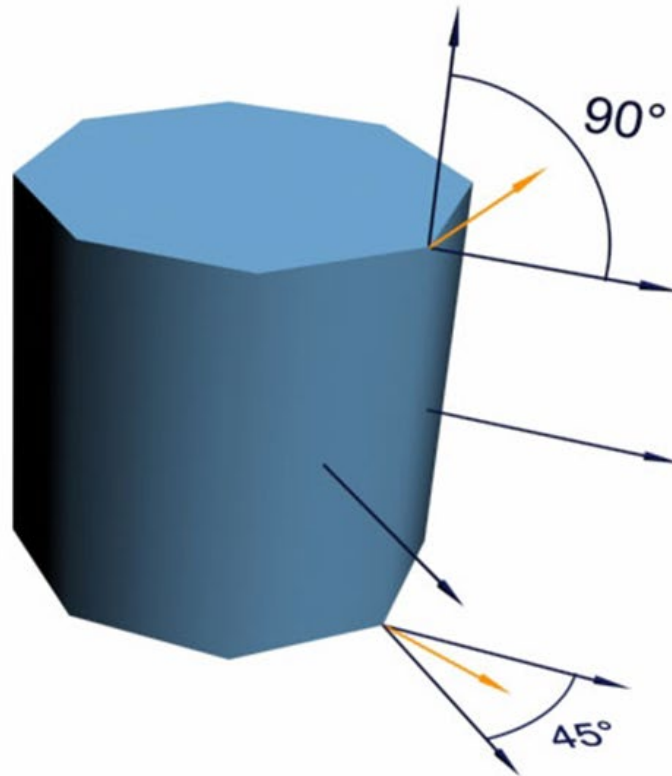
- Durch Eingabe eines Schwellenwertes (**smoothing-angle**), der mit dem Winkel zwischen zwei benachbarten Faces verglichen wird, kann das **Smoothing** beeinflusst werden. Liegt ein Winkel unter dem Schwellenwert, werden die betroffenen Polygone weich/smooth (z.B. Phong Shading) dargestellt, liegt der Winkel über dem Schwellenwert, werden die Flächen facettiert/flat (z.B. Flat Shading) dargestellt.

Smoothing

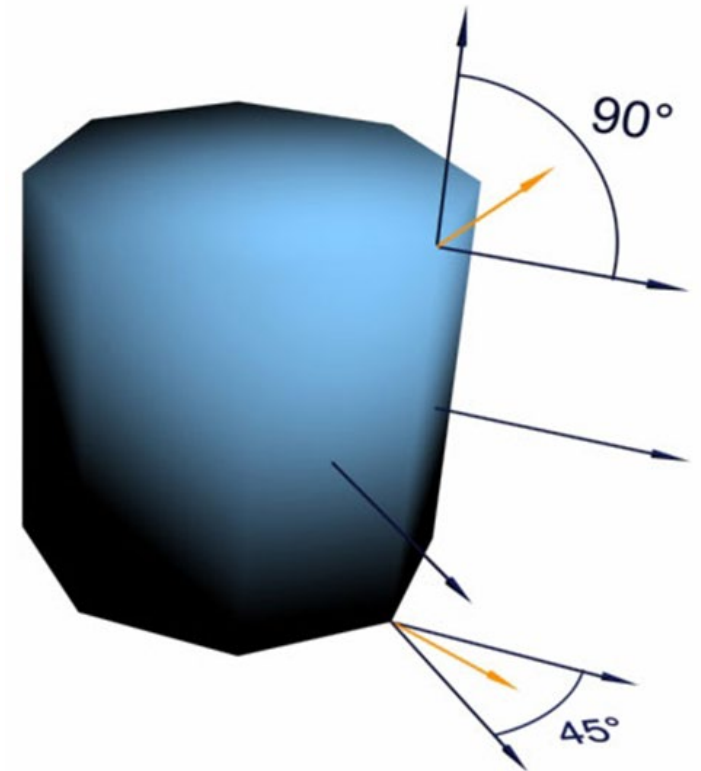
Smoothing
Winkel
 0°



Smoothing
Winkel
 45°



Smoothing
Winkel
 90°

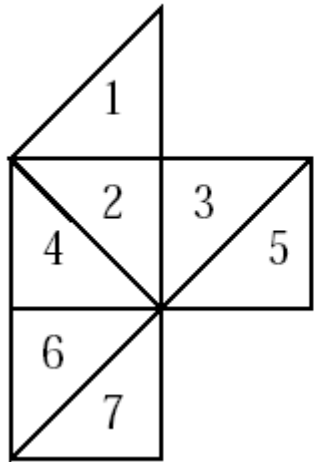


Greedy Algorithmus zur Erzeugung von Triangle Strips

Zusatzinformationen
für Übungen

In einem Hash wird für jedes Dreieck eine Liste seiner benachbarten Knoten gespeichert.

„Akeley, Haeberli und Burns ([AHB90]) haben einen gierigen Algorithmus zur Erzeugung von Triangle Strips vorgeschlagen. Als Startdreieck wird nach einem **Dreieck mit möglichst wenig Nachbarn** gesucht. Die Anzahl sollte natürlich größer als 0 sein; es ergibt keinen Sinn, ein isoliertes Dreieck als Startdreieck zu verwenden. Falls mehrere Dreiecke mit der gleichen minimalen Anzahl von Nachbarn existieren, wird das Dreieck ausgewählt, dessen Nachbar die minimale Anzahl von Nachbarn hat. Ist immer noch keine Entscheidung möglich, wird per Zufall ein Startdreieck ausgewählt. Dieses Dreieck und auch alle anderen im Verlauf des Algorithmus ausgewählten Dreiecke müssen natürlich aus dem Hash entfernt werden. Ausgehend vom Startdreieck wird nun das **Nachbardreieck ausgewählt, das wiederum die geringste Anzahl von Nachbarn hat**. Dies wird so lange fortgesetzt, bis keine Nachbarn mehr vorhanden sind. Aus den verbliebenen Dreiecken wird wiederum ein neues Startdreieck gewählt. Der Algorithmus stoppt dann, wenn der Hash leer ist, also alle Dreiecke zu einem Strip gehören. Ist der Algorithmus abgeschlossen können eventuell nochmals die Ecken reorganisiert werden, um korrekte Durchlaufrihtungen zu erzeugen.“
(aus Computergrafik, Bender&Brill, Hanser-Verlag 2006)

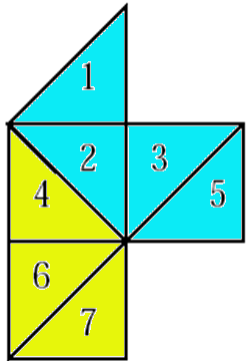


Nachbarn

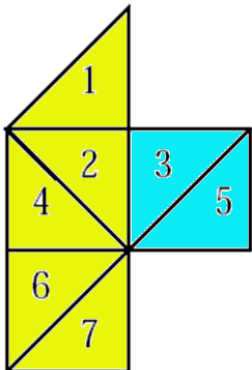
1	→	2		
2	→	1	3	4
3	→	2	5	
4	→	2	6	
5	→	3		
6	→	4	7	
7	→	6		

Beispiel

Zusatzinformationen
für Übungen



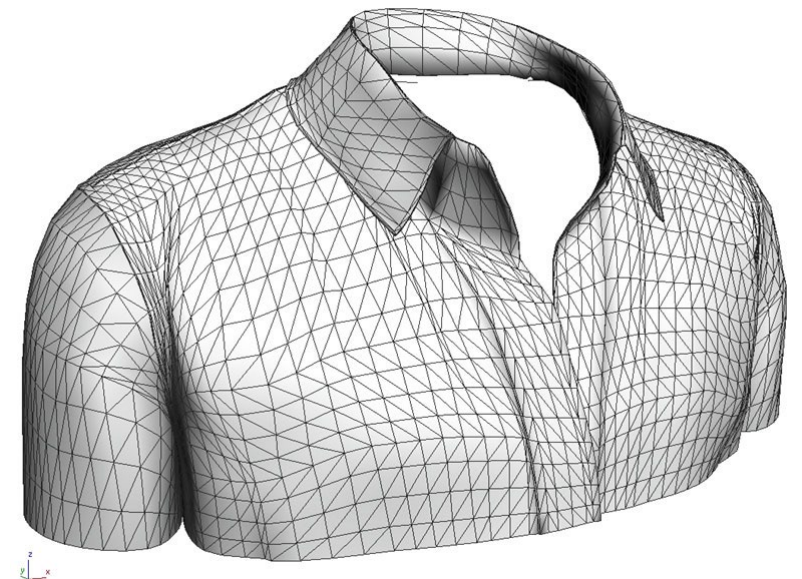
- Zuerst wird ein Strip mit den Dreiecken 5, 3, 2 und 1 berechnet. Dann startet der Algorithmus neu und legt einen zweiten Strip 4, 6 und 7 an.



- Oder: Zuerst 1,2,4,6 und 7 und dann 5 und 3.

TESSELATION

Perspective/Disabled



Tessellation

- **Die Polygone werden** bei der Tessellation **in primitive Flächen**, wie z.B. Dreiecke oder Vierecke **zerlegt**, da solche Flächen leichter zu handhaben sind als komplexe Polygone (insbesondere konkave).
- Spezialfall. **Dreieckszerlegung = Triangulation.**
- *tesselare (lat.) = mit Mosaik pflastern, mit einem Muster überziehen..*

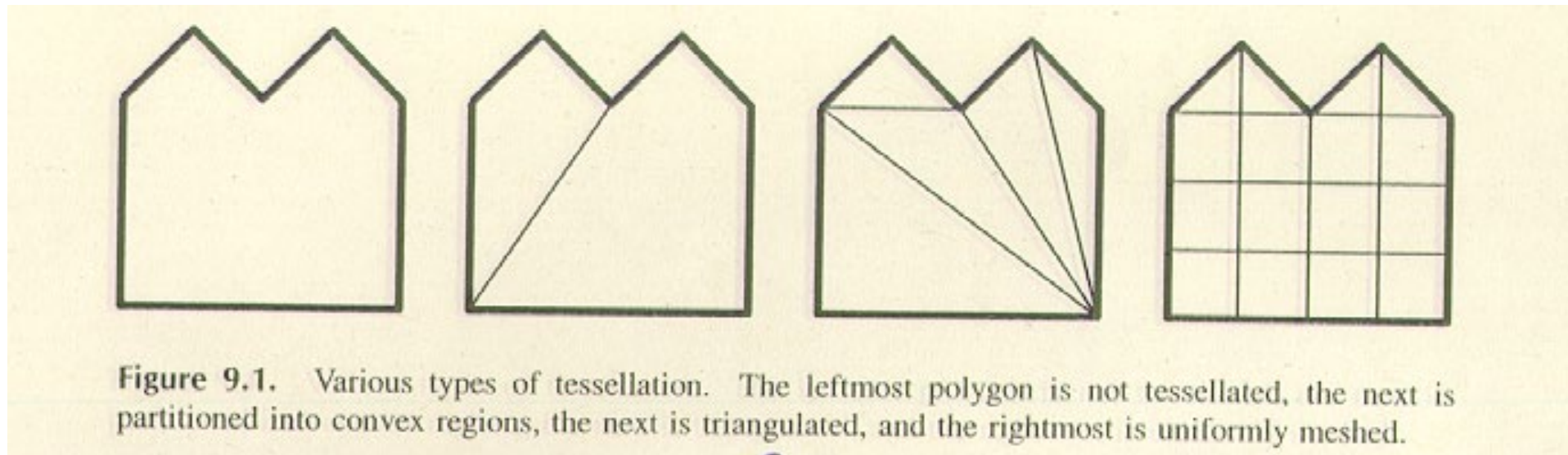
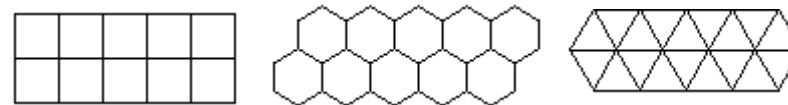
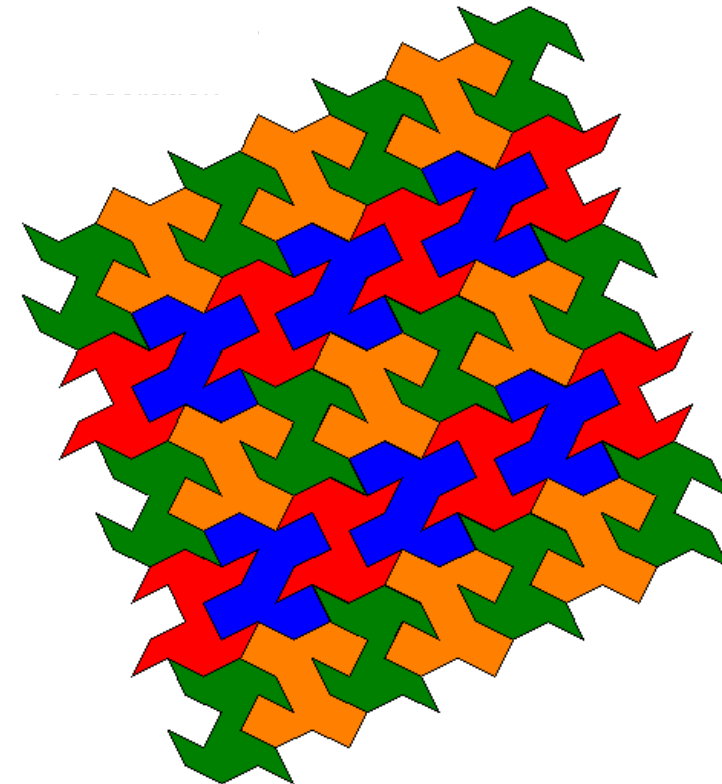
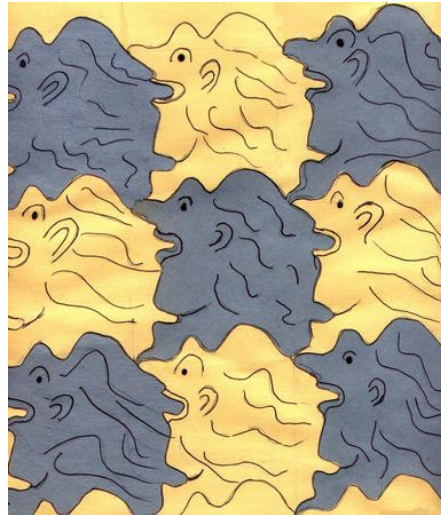
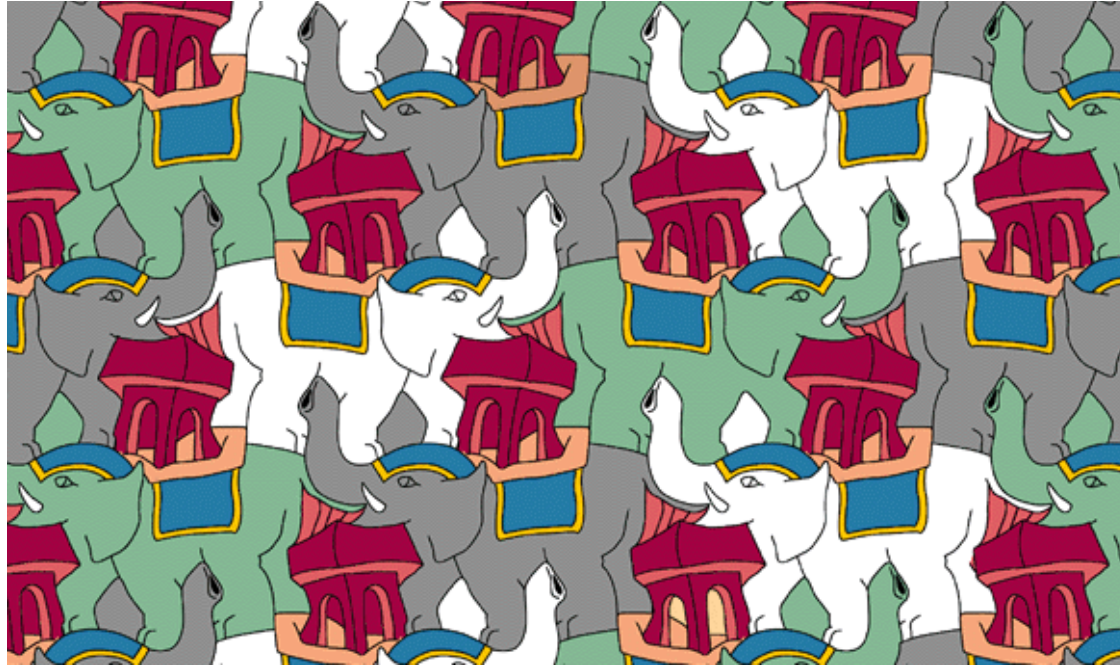


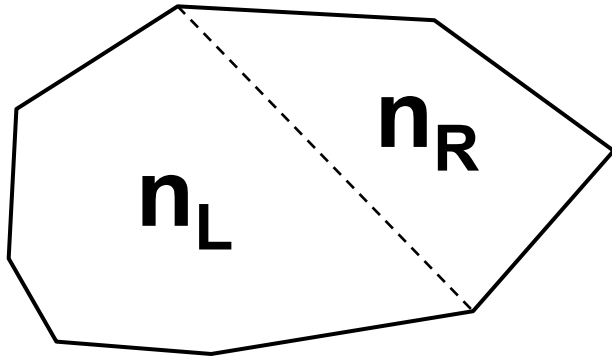
Figure 9.1. Various types of tessellation. The leftmost polygon is not tessellated, the next is partitioned into convex regions, the next is triangulated, and the rightmost is uniformly meshed.

Tessellation in der Kunst





**Wieviele Dreiecke
benötigt man minimal,
um ein einfaches Polygon
mit n Ecken zu kacheln?**



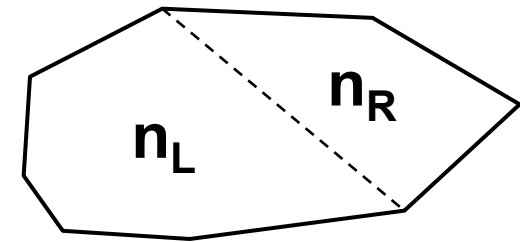
Triangulierung eines Polygons

Gegeben: ein einfaches Polygon P

Gesucht: Zerlegung von P in Dreiecke ohne Hinzufügung neuer Punkte

Frage: Wieviele Dreiecke braucht man ?

Behauptung: Sei n die Anzahl der Ecken des Polygons, dann ist die Anzahl der Dreiecke $t = n - 2$



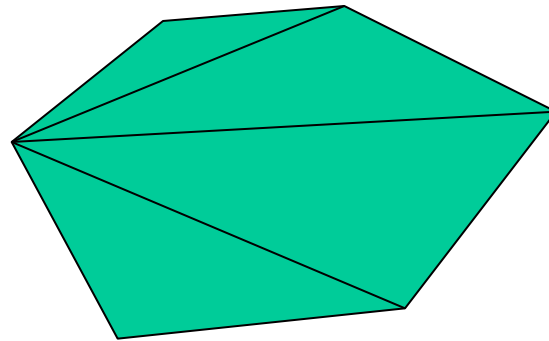
Beweis per Induktion:

I.V.: Polygon ist ein Dreieck ($n = 3$), dann ist $t = n - 2$

I.S.: Sei Triangulierung von P gegeben. Wir trennen P entlang einer Diagonalen (Dreieckskante, die im Inneren von P liegt). Dann erhalten wir zwei Polygon L und R mit $n_L + n_R = n + 2$ und wegen I.V. gilt $t_L = n_L - 2$ und $t_R = n_R - 2$. Die Gesamtzahl der Dreiecke ist dann: $t = t_R + t_L = (n_R - 2) + (n_L - 2) = (n_R + n_L) - 4 = (n + 2) - 4 = n - 2$

Triangulierung eines Polygons

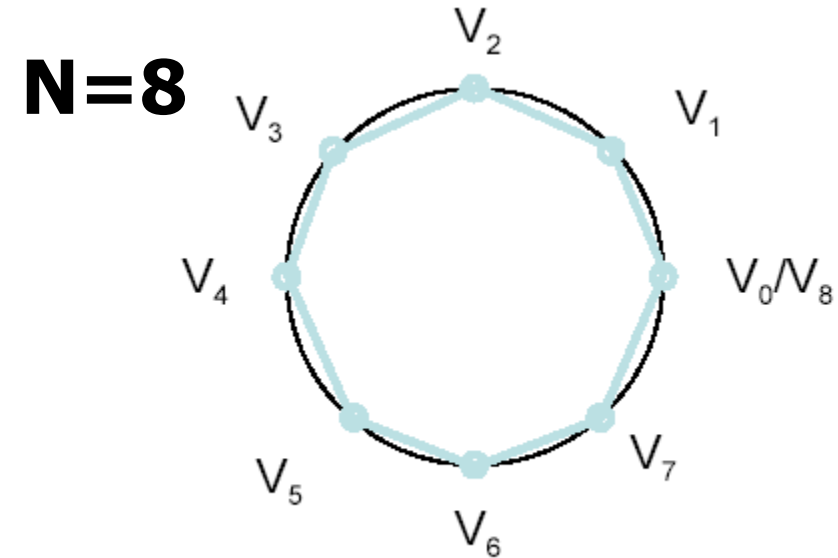
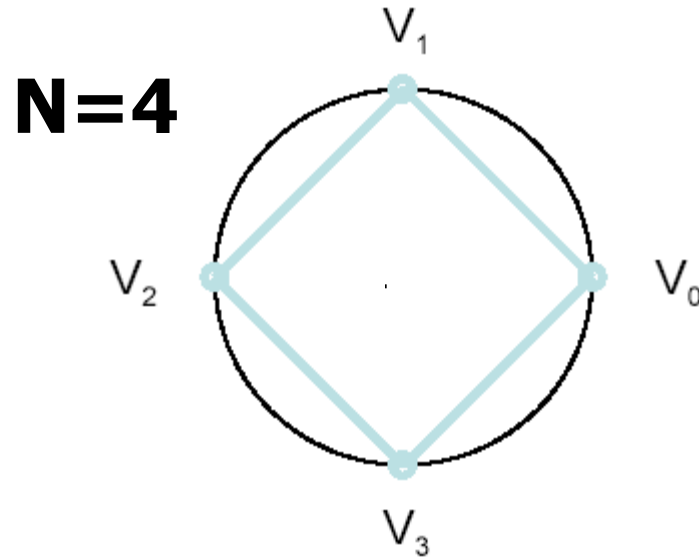
- für konvexe Polygone trivial:
 - Wähle eine Ecke und verbinde sie mit allen nicht benachbarten, anderen Ecken.



- für andere Arten von Polygonen → Vorlesung „Algorithmische Geometrie“ (S. Näher)

Approximieren eines Kreises in 2D

Wie lauten die Parameterwerte?



	t	x	y
• $t_0 = 0,$	0,00	1,00	0,00
• $t_1 = 1/4$	0,25	0,00	1,00
• $t_2 = 1/2$	0,50	-1,00	0,00
• $t_3 = 3/4$	0,75	0,00	-1,00
• $t_4 = 0.$	1,00	1,00	0,00

Annahme: Kreis mit Radius 1

Punkte auf der Kreislinie erfüllen:

$$x^2 + y^2 = 1 \quad (\rightarrow \text{Pythagoras})$$

Jeder Punkt ist durch t genau bestimmt:

$$x = \cos t * 2 * \pi$$

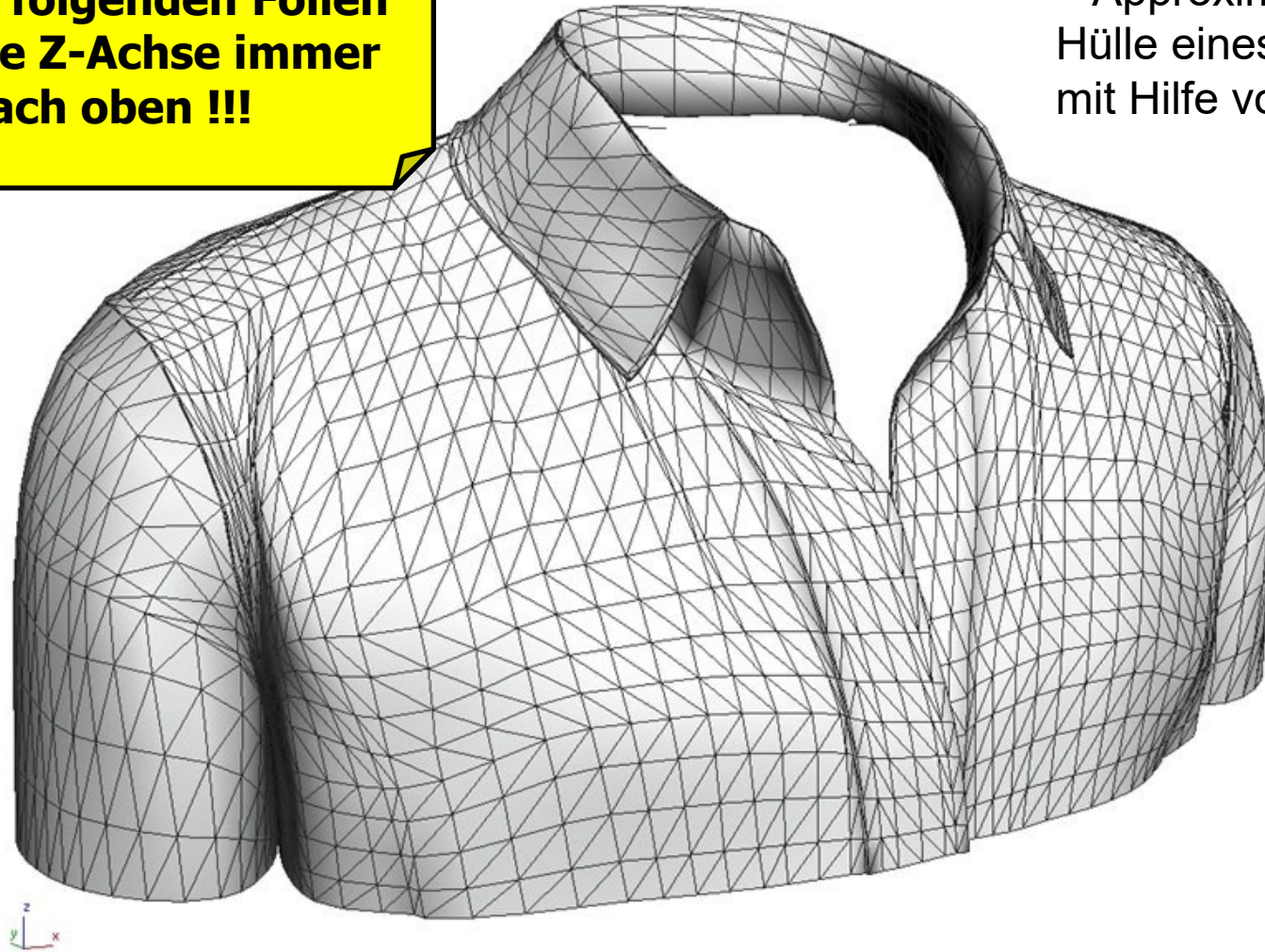
$$y = \sin t * 2 * \pi$$

Tessellation von Körpern

Perspective/Disabled

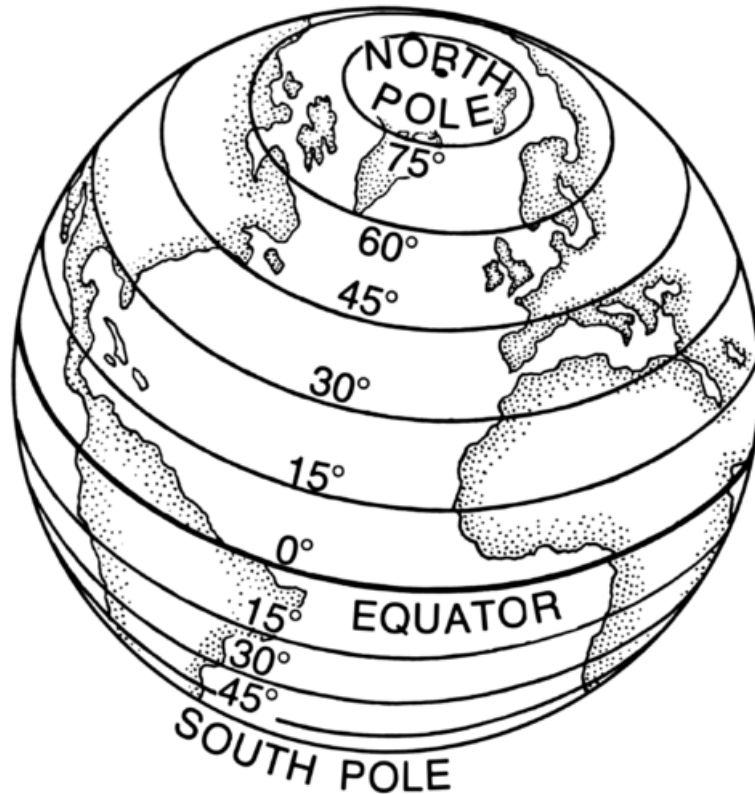
**Auf den folgenden Folien
zeigt die Z-Achse immer
nach oben !!!**

= Approximation der
Hülle eines Körpers
mit Hilfe von Polygonen.

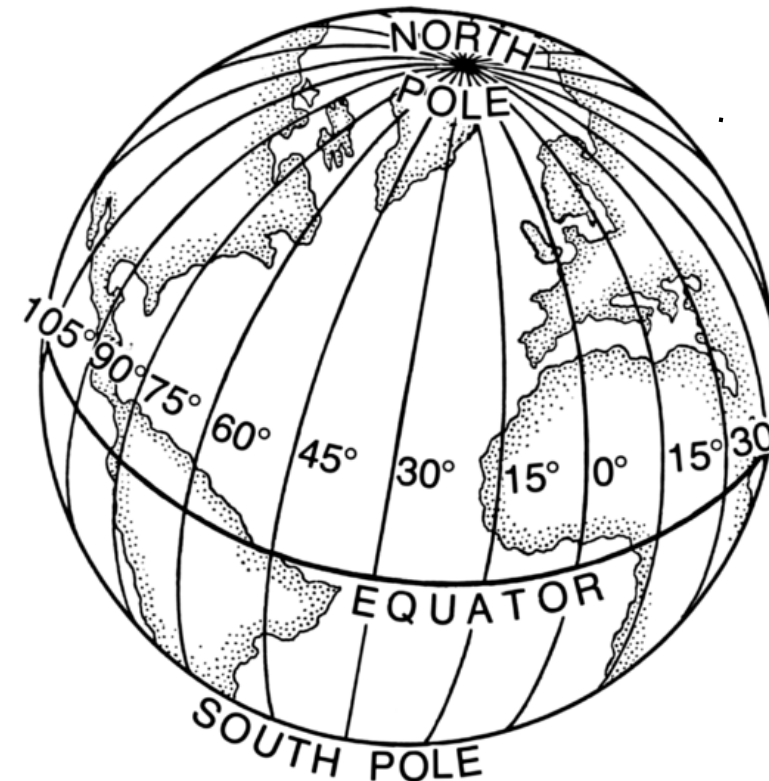


Geografische Koordinaten

Breitengrad (geografische Breite, Latitude)



Längengrad (geografische Länge, Longitude)



Kugel

Punkte der Kugeloberfläche erfüllen: $x^2 + y^2 + z^2 = 1$

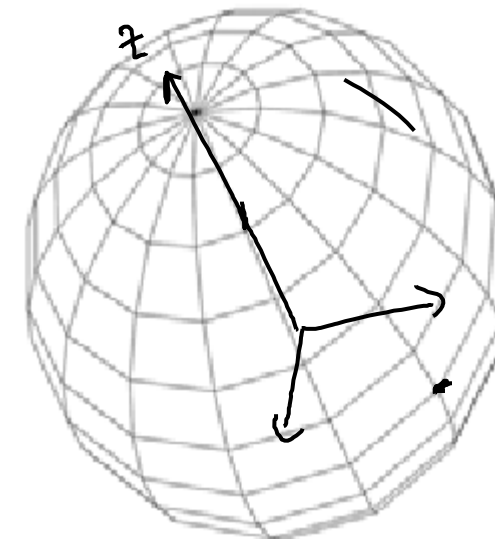
Kugelkoordinaten:

Kreis **Radius** **Längen-grad** **Breiten-grad**

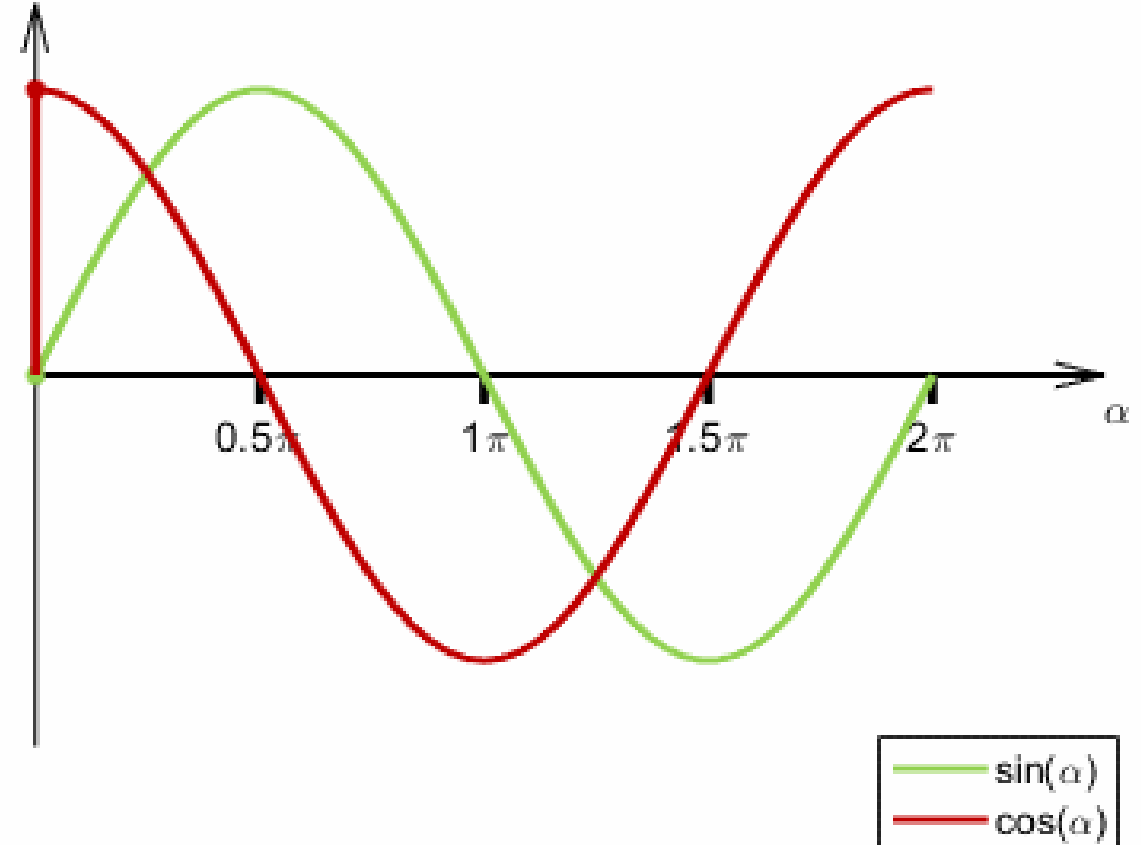
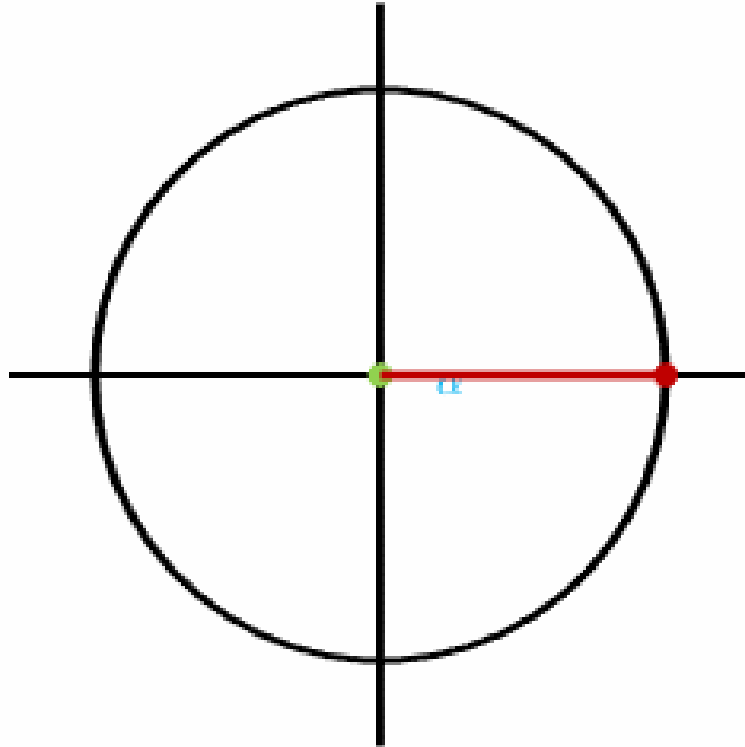
$$F(\underline{u}, \underline{v}) = \begin{pmatrix} \cos u * \cos v \\ \sin u * \cos v \\ \sin v \end{pmatrix} \text{ mit } u \in [0, 2\pi] \text{ und } v \in [-\frac{\pi}{2}, \frac{\pi}{2}]$$

Äquator: Breite $v = 0$,
Nordpol: Breite $v = \frac{\pi}{2}$
und Südpol: $v = -\frac{\pi}{2}$

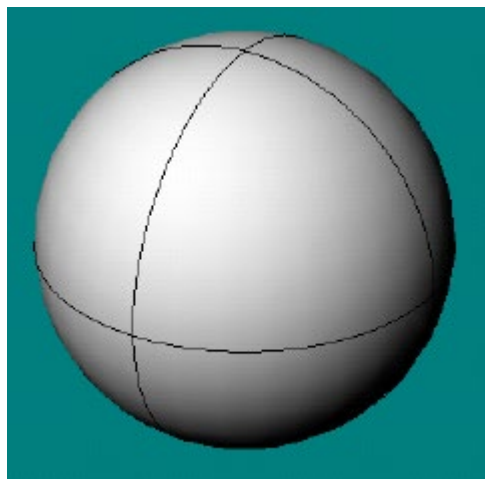
Höhe



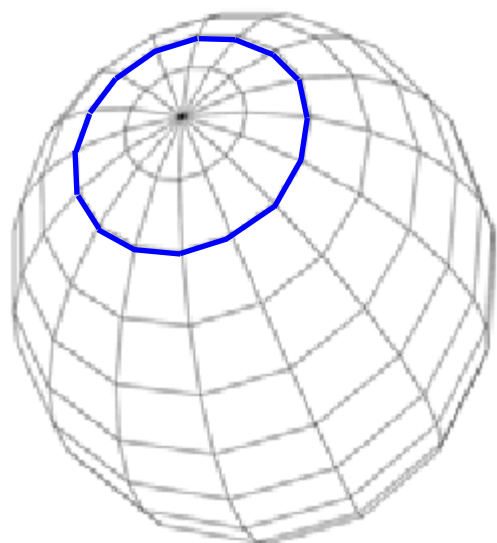
Sinus und Kosinus



Quelle: Wikipedia



$$u \in [0, 2\pi]$$

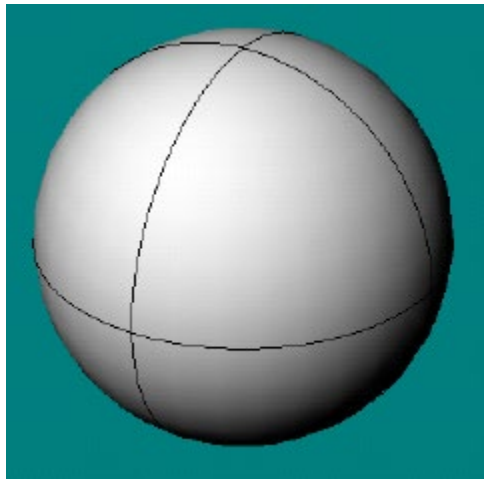


Breitenkreise

- Wir wählen eine **Auflösung für die Breitenkreise** und nennen diese n_B .
- Für die n_B Punkte

$$u_i = i(2\pi / n_B) \quad \text{mit } i = 0, \dots, n_B-1$$

erhalten wir eine Näherung der Breitenkreise durch einen Polygonzug.



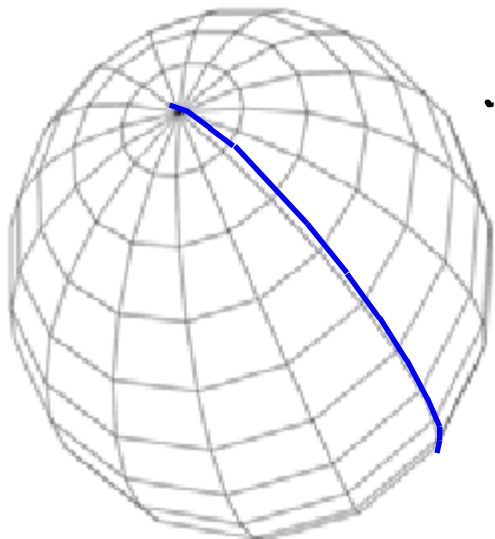
Längenkreise

- Zwischen den Polen zerschneiden wir die Kugel in eine Menge von Schichten.
- n_L soll die **Anzahl dieser Schichten** sein. Die erste und letzte Schicht enthalten nur einen Punkt, den jeweiligen Pol.
- Für die n_L Punkte

$$v \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$$

$$v_j = -\pi/2 + j(\pi/(n_L-1)) \quad \text{mit } j=0, \dots, n_L-1$$

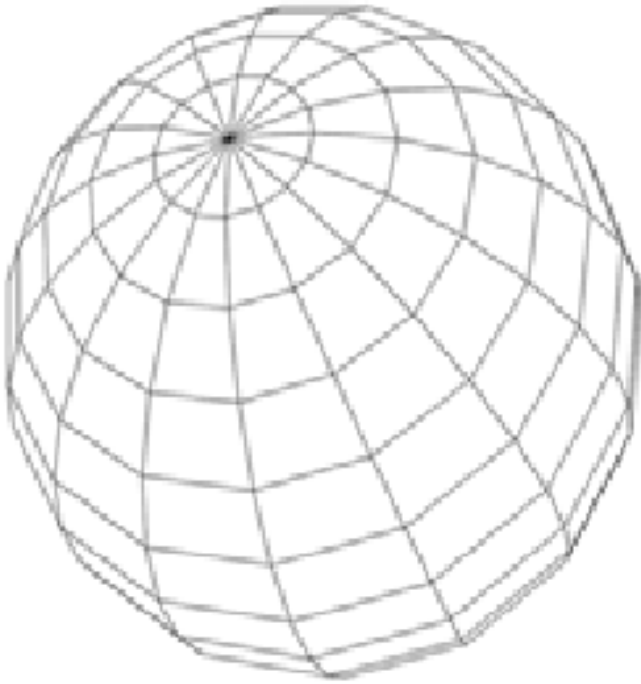
erhalten wir für festen Wert von u eine Näherung der Längenkreise durch Polygonzüge.



Eine Tessellation der Kugel

- Systematik für die Nummerierung der Punkte:
 - Der Nordpol ist V_0 ,
 - Dann folgen die n_B Punkte auf dem Breitenkreis unterhalb des Nordpols, bis wir am Südpol angekommen sind.
 - Wir verbinden die einzelnen Schichten; beispielsweise alle Punkte auf dem Breitenkreis unterhalb des Pols mit dem Pol.

Eine Tessellation der Kugel

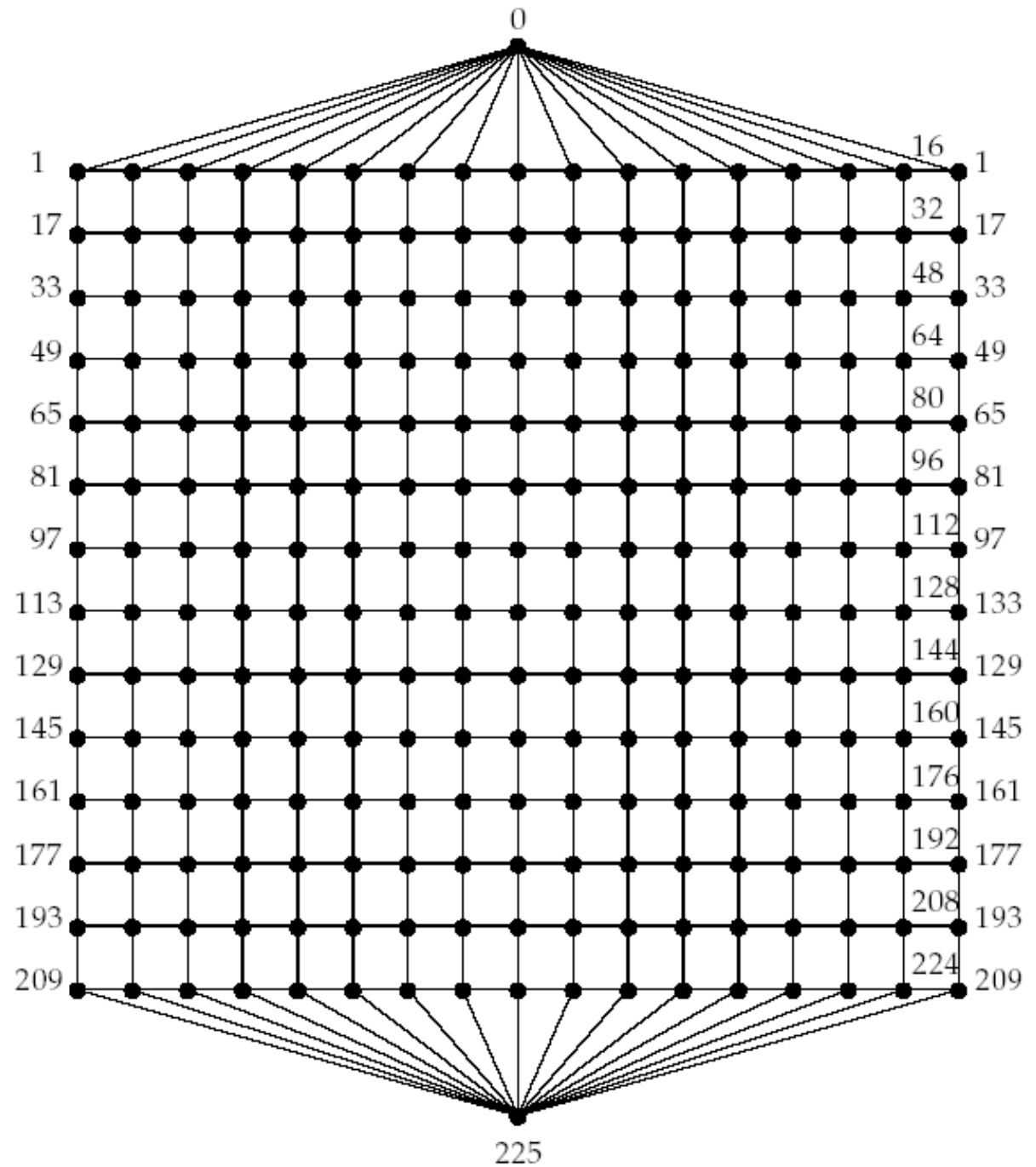
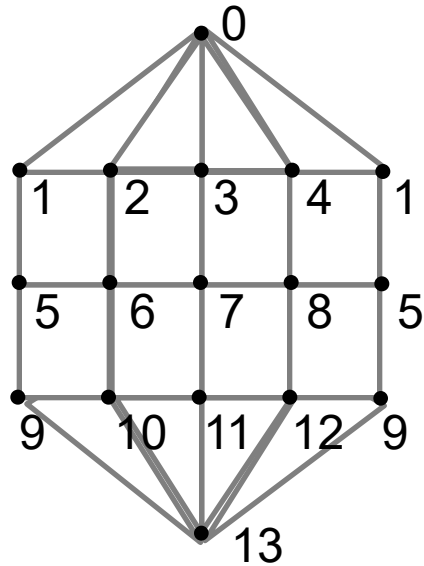
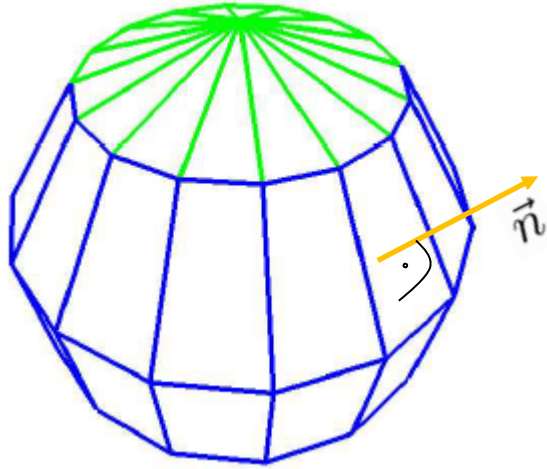


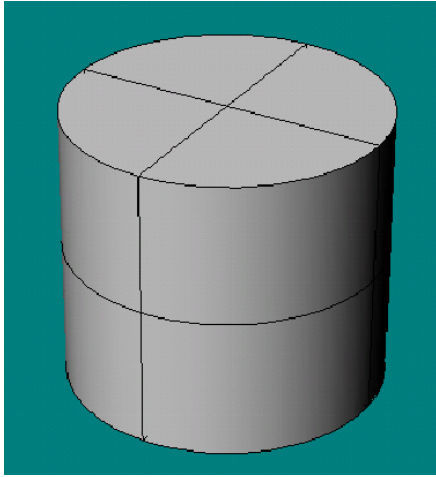
- Links sehen Sie das Ergebnis für $n_L = 12$, $n_B = 16$, mit 162 Punkten.
- Die Polregionen werden durch **Dreiecke** angenähert, der Rest durch **Vierecke**.
- Die **Eckennormalen** für dieses Netz werden durch den Verbindungsvektor der Ecken mit dem Mittelpunkt des Kreises gebildet.

Allgemein:

$(n_L - 2) * n_B + 2$ Eckpunkte

Tessellation der Kugel





Tessellation des Zylinders

Standardlage des Zylinders

- die Zylinderachse soll die z-Achse sein;
- der Radius ist 1,
- die Höhe soll von $z=0$ bis $z=1$ verlaufen.
- Unabhängig von den z-Werten ist ein Querschnitt parallel zur xy-Ebene, der Zylindermantel, immer durch einen Kreis gegeben.

Punkte des Zylindermantels erfüllen:

$$x^2 + y^2 = 1$$

Zylinderkoordinaten:

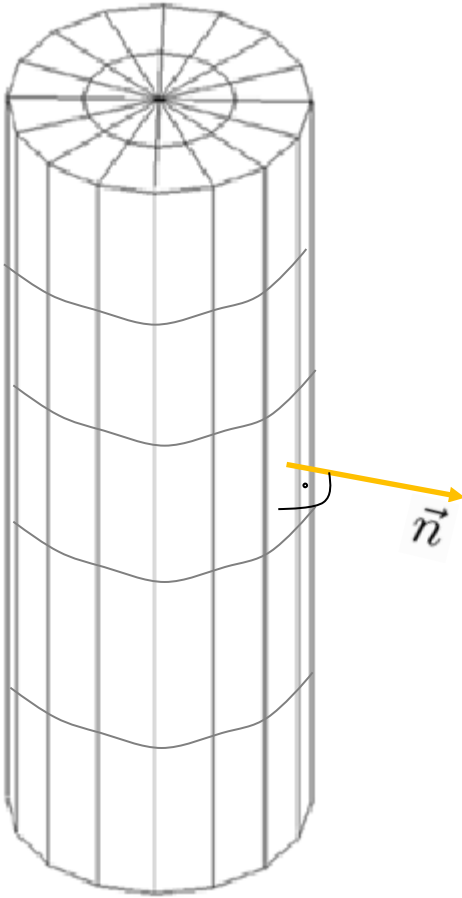
$$F(u, v) = \begin{pmatrix} \cos u \\ \sin u \\ v \end{pmatrix}$$

mit $u \in [0, 2\pi]$ und $v \in [0, 1]$

Tessellation des Zylinders

- Verfeinerung

- Wir zerlegen den Zylinder in Schichten, und nähern die Kreise wie gehabt an.
- Den Boden und den Deckel zerlegen wir ebenfalls in mehrere Kreise und bilden Vierecke wie links dargestellt.
- Im Zentrum des Bodens und des Deckels erhalten wir Dreiecke.
- Eckennormalen des Mantels sind $(\cos(u), \sin(u), 0)$



JS

*Folien für
Crash-Kurs
gibt es auf
der Webseite*

Geplanter Verlauf der Vorlesung

13.04.2026	Einleitung, Bild- und Videoformate
20.04.2026	Rastering, Zeichnen von Linien
27.04.2026	Transformationen + Projektionen
04.05.2026	SVG (Linien, Transformation, Animation)
11.05.2026	Beleuchtung
18.05.2026	Polygonnetze + Tessellation
25.05.2026	Pfingstmontag
01.06.2026	OpenGL (WebGL, Three.js)
08.06.2026	Shader in WebGL/Three.js
15.06.2026	Texture Mapping (Texturen in OpenGL als Aufgabe der Übung)
22.06.2026	Radiosity + Raytracing
29.06.2026	Animation (keyframing) + Rest (Kinematik, Physik, Simulation, Partikel, Schwärme)
06.07.2026	Wissenschaftliche Visualisierung, Marching Cubes
13.07.2026	Klausur

